MODUL 5

DIGITAL ENCODING DAN LINE CODING

Memahami NRZ, RZ, Manchester, Differential Manchester, dan Visualisasi Bit dengan Python

Mata Kuliah : Komunikasi Data
Program Studi : Teknik Informatika
Semester : 3
Bobot : 2 SKS
Model Pembelajaran : Concept Exploration, Guided Coding, Simulation, Case Study, dan Project-Based Learning
Perangkat Praktikum : Python melalui Google Colab/Jupyter Notebook

1. Deskripsi dan Capaian Pembelajaran

Pada Modul 4 kita telah memahami bahwa data dibawa oleh sinyal melalui media transmisi. Akan tetapi, masih terdapat satu pertanyaan penting: **bagaimana pola bit digital seperti 10110100 diterjemahkan menjadi bentuk sinyal yang dapat dikirim melalui media?**

Bit 0 dan 1 merupakan representasi logis. Agar dapat ditransmisikan, bit tersebut perlu dipetakan ke dalam pola sinyal tertentu. Proses mengubah data digital menjadi sinyal digital disebut **digital-to-digital conversion**, dan salah satu mekanisme utamanya adalah **line coding**.

Line coding menentukan bagaimana setiap bit atau kelompok bit direpresentasikan melalui perubahan level sinyal selama interval waktu tertentu. Pemilihan teknik line coding bukan hanya persoalan menentukan bahwa bit 1 diberi level tinggi dan bit 0 diberi level rendah. Sistem juga perlu mempertimbangkan kemampuan receiver dalam mempertahankan sinkronisasi, perubahan level sinyal, penggunaan bandwidth, dan karakteristik kanal.

Pada modul ini Anda akan mempelajari beberapa teknik line coding dasar, yaitu **NRZ-L, NRZ-I, RZ, Manchester**, dan **Differential Manchester**. Fokus utama bukan menghafalkan bentuk gelombang, tetapi memahami **aturan encoding** yang menghasilkan bentuk sinyal tersebut.

Python digunakan untuk membangun visualisasi bit dan menghasilkan line coding secara bertahap. Anda akan melihat bahwa satu pola bit yang sama dapat menghasilkan bentuk sinyal berbeda jika aturan encoding yang digunakan berbeda.

Setelah menyelesaikan modul ini Anda diharapkan mampu:

1. menjelaskan tujuan line coding dalam komunikasi digital;
2. membedakan data digital dan sinyal digital;
3. menjelaskan prinsip dasar NRZ-L, NRZ-I, RZ, Manchester, dan Differential Manchester;
4. mengidentifikasi kelebihan dan keterbatasan sederhana setiap teknik;
5. menjelaskan hubungan transisi sinyal dengan sinkronisasi;
6. memvisualisasikan bit menggunakan Python;
7. membuat fungsi sederhana untuk menghasilkan line coding; dan
8. menambahkan fitur **Line Coding Visualizer** pada Data Communication Simulator.

2. Mengapa Bit Perlu Di-encode?

Pada Modul 2 kita menggunakan pola seperti:

```
01000001
```

untuk merepresentasikan data digital. Pada Modul 3 kita kemudian melihat bahwa data harus dibawa oleh sinyal. Artinya, sender membutuhkan aturan untuk menentukan bentuk sinyal ketika bit bernilai `0` dan ketika bit bernilai `1`.

Bayangkan kita menggunakan aturan paling sederhana:

- bit `1` direpresentasikan dengan level positif;
- bit `0` direpresentasikan dengan level nol atau negatif.

Aturan tersebut tampak mudah, tetapi muncul persoalan ketika data berisi banyak bit dengan nilai yang sama secara berturut-turut.

Misalnya:

```
11111111
```

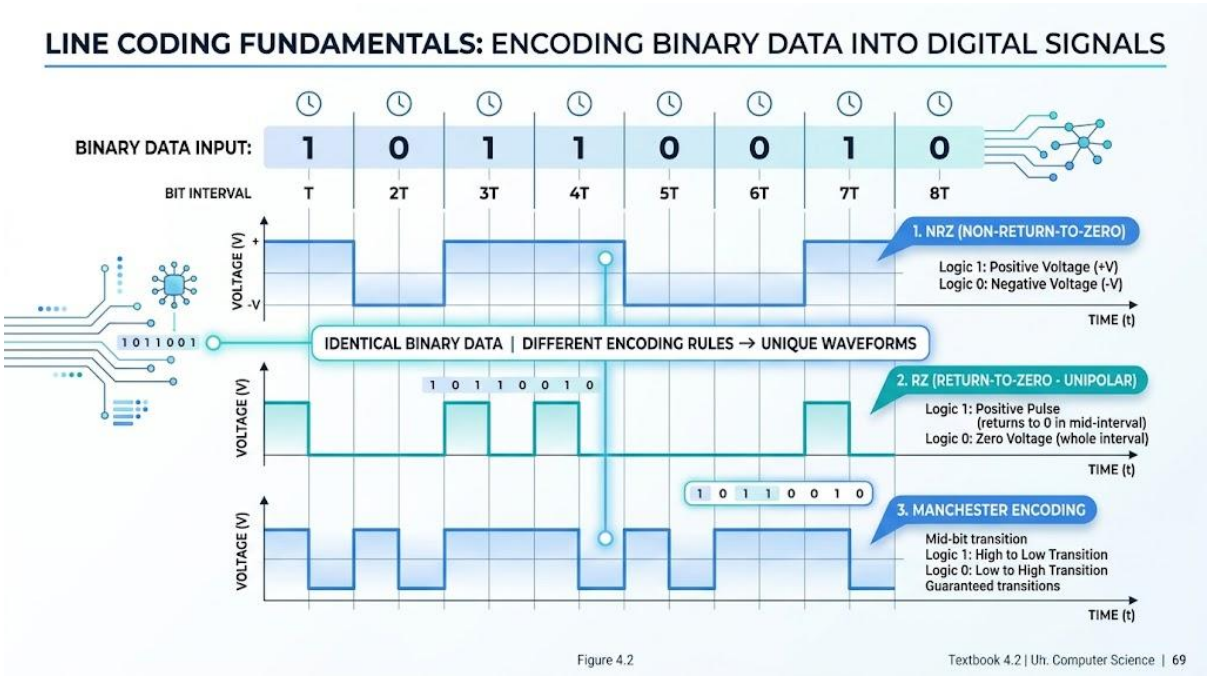
Jika level sinyal tidak berubah selama delapan interval bit, receiver perlu tetap mengetahui dengan tepat batas antara bit pertama, kedua, ketiga, dan seterusnya. Jika clock sender dan receiver tidak benar-benar selaras, kesalahan penentuan batas bit dapat terjadi.

Karena itu line coding tidak hanya berfungsi untuk membedakan `0` dan `1`. Teknik encoding juga dapat membantu menyediakan **transisi sinyal** yang berguna dalam proses sinkronisasi.

Dalam praktiknya, sebuah teknik line coding dapat dinilai dari beberapa aspek seperti jumlah transisi, komponen DC, kebutuhan bandwidth, kemampuan sinkronisasi, dan kompleksitas implementasi.

Pada modul ini aspek tersebut diperkenalkan secara konseptual. Analisis spektrum yang lebih kompleks belum diperlukan.

Referensi teori: Forouzan (2013), bagian Digital-to-Digital Conversion; Stallings (2022), bagian Digital Data, Digital Signals.



Gambar 5.1 — Data Digital Menjadi Sinyal Digital

3. Konsep Level Sinyal dan Transisi

Sebelum membahas setiap teknik, terdapat dua istilah yang perlu dipahami, yaitu **level** dan **transisi**.

Level sinyal menunjukkan keadaan sinyal pada suatu interval. Dalam model sederhana kita dapat menggunakan:

`+1` sebagai level positif,

`0` sebagai level nol,

dan `-1` sebagai level negatif.

Nilai tersebut tidak harus dianggap sebagai tegangan nyata. Dalam simulasi Python, angka tersebut hanya digunakan sebagai representasi logis agar bentuk sinyal mudah divisualisasikan.

Sementara itu, **transisi** terjadi ketika level sinyal berubah dari satu keadaan ke keadaan lainnya.

Contohnya, perubahan dari:

`+1` menjadi `-1`

merupakan satu transisi.

Transisi sangat penting karena dapat memberikan petunjuk waktu kepada receiver. Teknik yang secara teratur menghasilkan transisi biasanya mempunyai kemampuan sinkronisasi lebih baik daripada teknik yang dapat mempertahankan level yang sama dalam waktu panjang.

4. NRZ — Non-Return-to-Zero

NRZ merupakan salah satu konsep line coding paling sederhana. Istilah **Non-Return-to-Zero** menunjukkan bahwa selama satu interval bit, sinyal tidak harus kembali ke level nol di tengah interval.

Terdapat beberapa variasi NRZ. Pada modul ini kita membahas **NRZ-L** dan **NRZ-I**.

4.1 NRZ-L

Pada **NRZ-Level**, nilai bit direpresentasikan langsung oleh level sinyal tertentu.

Untuk keperluan modul ini kita menggunakan konvensi:

- bit `1` = level `+1`;
- bit `0` = level `-1`.

Perlu diperhatikan bahwa beberapa literatur atau standar dapat menggunakan polaritas sebaliknya. Yang penting adalah sender dan receiver mengikuti aturan yang sama.

Jika data:

`10110`

maka setiap interval bit diberikan level berdasarkan nilai bitnya.

NRZ-L mudah dibuat dan mudah dipahami. Namun apabila terdapat rangkaian bit yang sama dalam jumlah panjang, sinyal dapat tidak mengalami transisi untuk waktu yang cukup lama. Kondisi tersebut dapat menyulitkan sinkronisasi.

4.2 NRZ-I

Pada **NRZ-Invert**, informasi tidak hanya ditentukan oleh level absolut, tetapi oleh apakah terjadi perubahan level.

Dalam salah satu konvensi yang umum digunakan:

- bit `1` menyebabkan perubahan level pada awal interval;
- bit `0` tidak menyebabkan perubahan level.

Misalnya sistem memulai dari level negatif. Ketika bit pertama adalah `1`, level berubah menjadi positif. Jika bit berikutnya `0`, level tetap. Jika bit berikutnya `1`, level kembali berubah.

Dengan demikian NRZ-I memperkenalkan gagasan penting bahwa **informasi dapat direpresentasikan oleh perubahan sinyal**, bukan hanya nilai level sinyal itu sendiri.

Referensi teori: Forouzan (2013); Stallings (2022).

5. Praktikum Python 1 — Visualisasi NRZ-L

Kita mulai dengan teknik yang paling sederhana.

Kode Program 5.1

```
import matplotlib.pyplot as plt

bits = [1, 0, 1, 1, 0, 0, 1]

levels = []

for bit in bits:
    if bit == 1:
        levels.append(1)
    else:
        levels.append(-1)

plt.step(
    range(len(levels)),
    levels,
    where="post"
)

plt.xlabel("Interval Bit")
plt.ylabel("Level Sinyal")
plt.title("NRZ-L")

plt.ylim(-1.5, 1.5)
plt.grid()
plt.show()
```

Memahami Command

Command	**Penjelasan**
`bits = [...]`	Membuat list berisi data bit yang akan di-encode.
`levels = []`	Membuat list kosong untuk menyimpan level hasil encoding.
`for bit in bits:`	Melakukan perulangan untuk membaca setiap bit.
`if bit == 1:`	Memeriksa apakah nilai bit sama dengan 1.
`==`	Operator perbandingan “sama dengan”.

Command	Penjelasan
levels.append(1)	Menambahkan level +1 ke dalam list.
else:	Menjalankan perintah alternatif jika kondisi if tidak terpenuhi.
levels.append(-1)	Menambahkan level -1 untuk bit 0.
plt.step()	Membuat grafik bertingkat untuk data diskrit.

Python Corner — Mengenal for, if, dan .append()

Pada program ini Anda mulai menggunakan logika pemrograman yang lebih penting.

for digunakan untuk membaca bit satu per satu.

if digunakan untuk mengambil keputusan berdasarkan nilai bit.

.append() digunakan untuk menambahkan hasil ke bagian akhir sebuah list.

Program tersebut pada dasarnya menerapkan aturan:

> jika bit 1, tambahkan +1; jika bukan, tambahkan -1.

Inilah contoh langsung bagaimana **aturan line coding diterjemahkan menjadi algoritma**.

Coba Otak-Atik

Ubah pola:

```
bits = [1, 0, 1, 1, 0, 0, 1]
```

menjadi:

```
bits = [1, 1, 1, 1, 1, 1, 1]
```

Kemudian coba:

```
bits = [1, 0, 1, 0, 1, 0, 1]
```

6. RZ — Return-to-Zero

Pada teknik **Return-to-Zero**, sinyal tidak mempertahankan satu level selama seluruh interval bit. Dalam satu interval, sinyal kembali menuju level nol.

Secara konseptual, untuk setiap bit terdapat perubahan tambahan di dalam interval bit.

Sebagai contoh sederhana:

- bit `1` dapat direpresentasikan dengan level positif pada setengah awal interval, lalu kembali ke nol;
- bit `0` dapat direpresentasikan dengan level negatif pada setengah awal interval, lalu kembali ke nol.

Karena terdapat perubahan di dalam interval, receiver memperoleh lebih banyak transisi dibandingkan NRZ.

Namun konsekuensinya, RZ membutuhkan perubahan sinyal yang lebih sering dan secara umum memerlukan bandwidth lebih besar daripada NRZ.

RZ membantu Anda memahami bahwa desain line coding selalu melibatkan kompromi. Teknik yang memberikan lebih banyak transisi untuk sinkronisasi dapat membutuhkan sumber daya spektrum yang lebih besar.

Referensi teori: Forouzan (2013), bagian Polar Encoding.

7. Manchester Encoding

Manchester merupakan salah satu teknik line coding yang menggunakan **transisi pada pertengahan setiap interval bit**.

Keberadaan transisi di tengah interval membuat Manchester memiliki kemampuan *self-clocking* yang baik karena receiver memperoleh informasi timing secara teratur.

Dalam modul ini kita menggunakan konvensi:

- bit `0` = transisi dari level positif ke negatif;
- bit `1` = transisi dari level negatif ke positif.

Perlu diperhatikan bahwa konvensi dapat ditulis berbeda pada referensi tertentu. Karena itu aturan encoding harus selalu dinyatakan secara eksplisit.

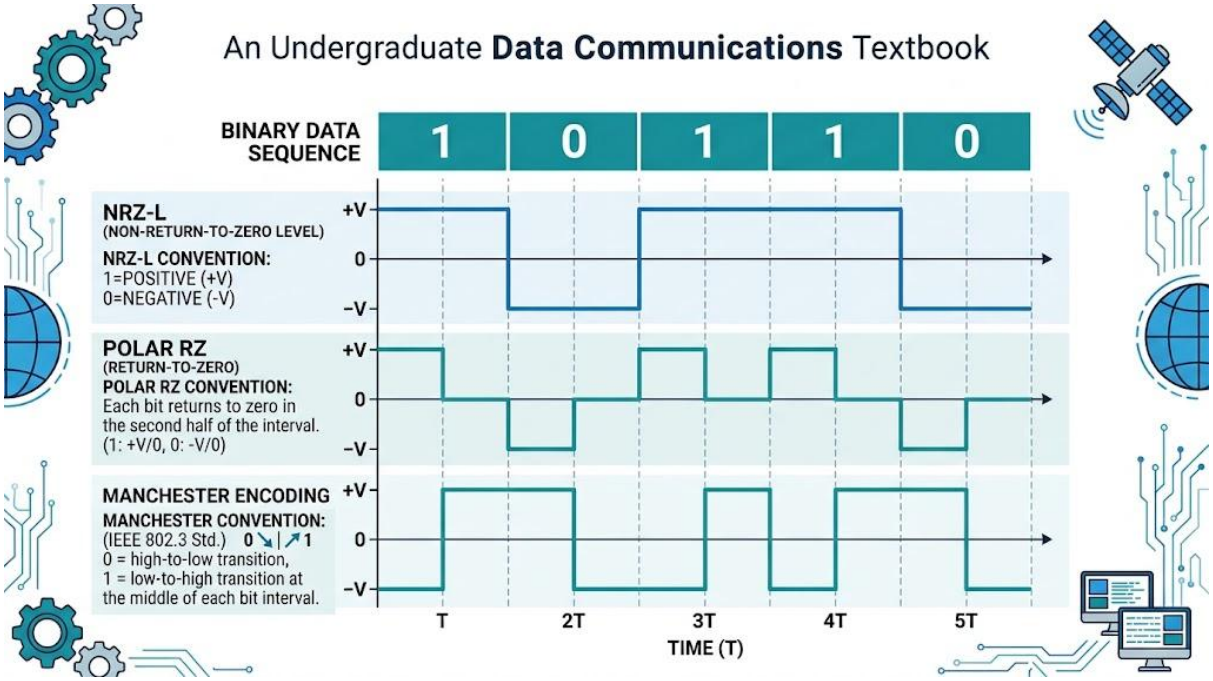
Jika bit `1` akan dikirim, setengah interval pertama berada pada level negatif, kemudian berubah ke positif pada pertengahan interval.

Jika bit `0` dikirim, pola berlaku sebaliknya.

Karena setiap bit selalu mempunyai transisi di tengah, deretan bit yang sama tetap menghasilkan perubahan sinyal. Ini memberikan keuntungan besar untuk sinkronisasi dibandingkan NRZ.

Namun jumlah transisi yang lebih banyak juga meningkatkan kebutuhan bandwidth.

Referensi teori: Forouzan (2013); Stallings (2022).



Gambar 5.2 — Perbandingan NRZ-L, RZ, dan Manchester

8. Praktikum Python 2 — Membuat Manchester Encoding

Karena setiap bit mempunyai dua bagian, kita akan membuat dua level untuk setiap satu bit.

Kode Program 5.2

```
import matplotlib.pyplot as plt

bits = [1, 0, 1, 1, 0]

manchester = []

for bit in bits:
    if bit == 1:
        manchester.extend([-1, 1])
    else:
        manchester.extend([1, -1])

plt.step(
    range(len(manchester)),
    manchester,
    where="post"
)

plt.xlabel("Setengah Interval Bit")
plt.ylabel("Level Sinyal")
plt.title("Manchester Encoding")

plt.ylim(-1.5, 1.5)
plt.grid()
plt.show()
```

Memahami Command

Command	**Penjelasan**
`manchester = []`	Membuat list kosong untuk menyimpan hasil encoding.
`for bit in bits:`	Membaca setiap bit satu per satu.
`manchester.extend([-1, 1])`	Menambahkan dua level sekaligus untuk bit 1.
`manchester.extend([1, -1])`	Menambahkan dua level untuk bit 0.

Command	Penjelasan
.extend()	Menambahkan beberapa elemen dari sebuah list ke list lainnya.

Perbedaan .append() dan .extend()

Pada NRZ-L kita menggunakan:

```
levels.append(1)
```

karena satu bit hanya menghasilkan satu nilai level dalam model sederhana.

Pada Manchester, satu bit menghasilkan dua bagian sehingga kita menggunakan:

```
manchester.extend([-1,                                                    1])
```

.extend() menambahkan kedua nilai secara terpisah ke dalam list.

Coba Ubah

Gunakan pola:

```
bits = [1, 1, 1, 1, 1]
```

Kemudian bandingkan dengan hasil NRZ-L untuk data yang sama.

Pertanyaan

1. Teknik mana yang tetap mempunyai transisi meskipun semua bit bernilai sama?
2. Mengapa Manchester lebih mudah membantu receiver mempertahankan sinkronisasi?
3. Apa konsekuensi dari semakin banyaknya transisi?

9. Differential Manchester

Differential Manchester juga menggunakan transisi pada pertengahan setiap interval bit. Transisi tersebut digunakan untuk membantu sinkronisasi.

Perbedaannya, informasi bit ditentukan oleh **ada atau tidak adanya transisi tambahan pada awal interval**, bukan oleh arah absolut transisi di tengah.

Untuk modul ini digunakan konvensi:

- bit `0` = terdapat transisi pada awal interval;
- bit `1` = tidak terdapat transisi pada awal interval;
- selalu terdapat transisi pada pertengahan interval.

Keuntungan pendekatan diferensial adalah informasi lebih bergantung pada perubahan level daripada polaritas absolut. Jika polaritas sinyal terbalik selama transmisi, interpretasi transisi relatif masih dapat digunakan.

Namun algoritmanya sedikit lebih kompleks dibandingkan NRZ-L atau Manchester biasa karena hasil encoding suatu bit juga dipengaruhi oleh level sebelumnya.

Referensi teori: Forouzan (2013); Stallings (2022).

10. Praktikum Python 3 — Differential Manchester

Kode Program 5.3

```
import matplotlib.pyplot as plt

bits = [1, 0, 1, 1, 0]

signal = []
current_level = 1

for bit in bits:

    if bit == 0:
        current_level = -current_level

    signal.append(current_level)

    current_level = -current_level

    signal.append(current_level)

plt.step(
    range(len(signal)),
    signal,
    where="post"
)

plt.xlabel("Setengah Interval Bit")
plt.ylabel("Level Sinyal")
plt.title("Differential Manchester")

plt.ylim(-1.5, 1.5)
plt.grid()
plt.show()
```

Memahami Command

Command	**Penjelasan**
`current_level = 1`	Menentukan level awal sinyal.
`if bit == 0:`	Memeriksa apakah bit membutuhkan transisi awal.
`current_level = -current_level`	Membalik level sinyal, misalnya +1 menjadi -1.

Command	Penjelasan
signal.append(current_level)	Menyimpan level pada setengah interval pertama.
current_level = -current_level	Membuat transisi wajib di pertengahan interval.
signal.append(current_level)	Menyimpan level pada setengah interval kedua.

Mengapa Ada current_level?

Pada NRZ-L, level dapat ditentukan hanya berdasarkan nilai bit yang sedang dibaca.

Pada Differential Manchester, encoding juga dipengaruhi **kondisi level sebelumnya**.

Variabel:

```
current_level
```

digunakan untuk mengingat keadaan sinyal saat ini.

Konsep “menyimpan state” seperti ini nantinya akan banyak muncul dalam simulasi komunikasi data.

11. Membandingkan Teknik Line Coding

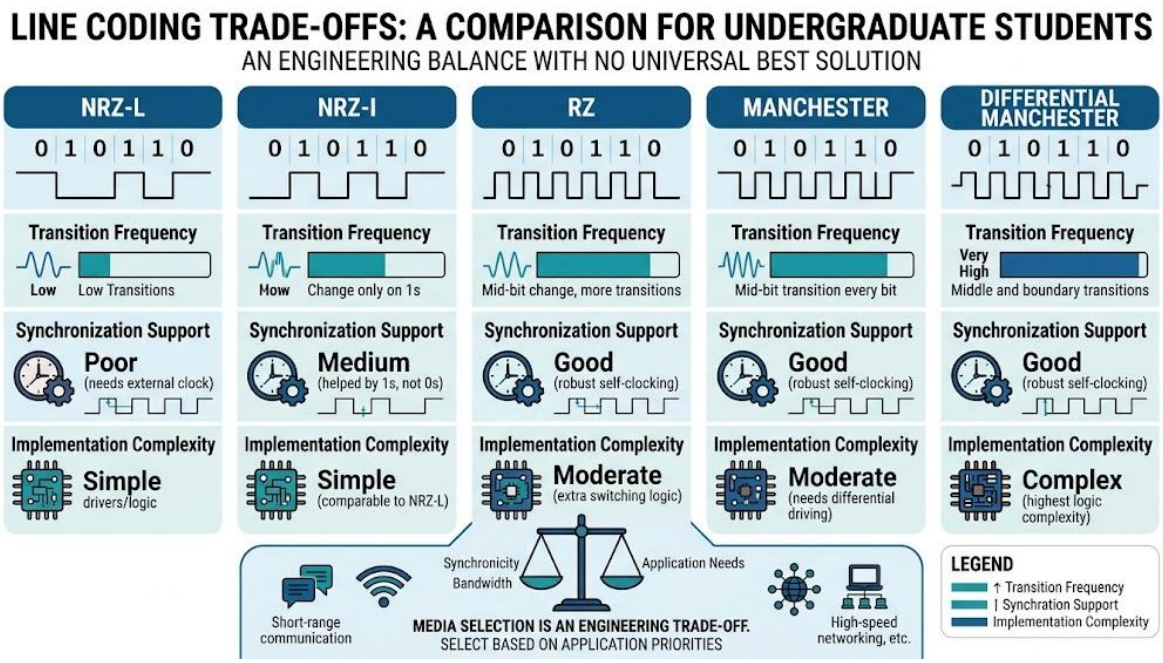
Tidak ada satu teknik yang selalu paling baik untuk seluruh sistem.

Perbandingan sederhana dapat dilihat berikut.

Teknik	**Prinsip Utama**	**Transisi**	**Sinkronisasi**
NRZ-L	Bit ditentukan level	Bisa sedikit	Relatif lemah pada deretan bit sama
NRZ-I	Bit tertentu menyebabkan perubahan	Tergantung data	Lebih baik pada pola tertentu
RZ	Sinyal kembali ke nol	Lebih banyak	Lebih baik dari NRZ
Manchester	Transisi wajib di tengah bit	Banyak	Baik
Differential Manchester	Transisi tengah + aturan diferensial	Banyak	Baik

Tabel tersebut merupakan perbandingan konseptual, bukan penilaian lengkap mengenai seluruh parameter performa.

Saat menentukan line coding nyata, engineer juga harus mempertimbangkan bandwidth, komponen DC, kompleksitas perangkat, kemampuan error detection tertentu, dan kebutuhan sistem.



Gambar 5.3 — Trade-off Line Coding

12. Studi Kasus — Sinkronisasi pada Data Sensor

Sebuah sistem sensor menghasilkan status digital setiap interval tertentu.

Pada suatu periode, sensor menghasilkan pola:

`11111111`

Tim pengembang menggunakan teknik NRZ-L dengan bit `1` direpresentasikan sebagai level tinggi.

Akibatnya, selama seluruh deretan bit tersebut, sinyal dapat terlihat sebagai satu level yang tidak berubah.

Receiver tidak hanya harus mengetahui apakah level tersebut mewakili `1`, tetapi juga harus mempertahankan clock yang cukup akurat agar dapat menentukan bahwa level panjang tersebut sebenarnya terdiri atas delapan bit terpisah.

Sekarang tim mencoba Manchester encoding.

Walaupun seluruh data tetap:

`11111111`

setiap bit memiliki transisi di tengah interval. Receiver memperoleh perubahan sinyal secara teratur yang dapat membantu sinkronisasi.

Tugas Praktikum

Gunakan:

```
bits = [1, 1, 1, 1, 1, 1, 1, 1]
```

Visualisasikan menggunakan:

1. NRZ-L;
2. Manchester.

Kemudian bandingkan hasilnya.

Pertanyaan Analisis

1. Pada NRZ-L, berapa banyak perubahan level yang terlihat pada pola tersebut?
2. Mengapa receiver dapat lebih sulit menentukan batas bit pada rangkaian panjang tanpa transisi?
3. Apa yang berbeda pada Manchester?
4. Mengapa peningkatan kemampuan sinkronisasi dapat mempunyai konsekuensi terhadap kebutuhan bandwidth?

13. Project Semester — Milestone 5

Pada Modul 1 simulator mempunyai **Communication Profile**.

Modul 2 menambahkan **Message Encoder**.

Modul 3 menambahkan **Signal Visualizer**.

Modul 4 menambahkan **Transmission Channel Analyzer**.

Pada Modul 5 ditambahkan:

Line Coding Visualizer

Anda membuat fitur yang menerima bit dan memilih teknik encoding untuk divisualisasikan.

Tahap Awal

Gunakan pola bit:

```
bits = [1, 0, 1, 1, 0, 1]
```

Kemudian buat fungsi NRZ-L.

```
def encode_nrz_l(bits):

    levels = []

    for bit in bits:

        if bit == 1:
            levels.append(1)
        else:
            levels.append(-1)

    return levels
```

Command Baru: `def`

`def` digunakan untuk membuat **fungsi sendiri**.

Pada:

```
def                                                    encode_nrz_l(bits):
```

kita membuat fungsi bernama:

`encode_nrz_l`

yang menerima input berupa:

`bits`.

Perintah:

```
return levels
```

mengembalikan hasil encoding kepada bagian program yang memanggil fungsi.

Menggunakan Fungsi

```
encoded = encode_nrz_l(bits)

print(encoded)
```

Anda mulai belajar bahwa program besar sebaiknya dibagi menjadi fungsi-fungsi yang mempunyai tugas tertentu.

Pengembangan Project

Buat minimal dua fungsi:

```
encode_nrz_l()
```

dan:

```
encode_manchester()
```

Kemudian gunakan input:

```
data = input("Masukkan bit: ")
```

Misalnya:

`101101`

Agar setiap karakter dapat diproses sebagai integer, dapat digunakan:

```
bits = [int(bit) for bit in data]
```

Command Baru: List Comprehension

Baris:

```
bits = [int(bit) for bit in data]
```

merupakan bentuk ringkas untuk membaca setiap karakter pada string, mengubahnya menjadi integer, kemudian memasukkannya ke dalam list.

Jika:

```
data = "1011"
```

hasilnya:

```
[1, 0, 1, 1]
```

Challenge Project

Program meminta pengguna memilih:

`1 = NRZ-L`

`2 = Manchester`

Kemudian menampilkan hasil encoding dan grafik.

Anda belum harus membuat antarmuka grafis. Pemilihan cukup dilakukan menggunakan `input()` dan `if`.

Advanced Challenge

Tambahkan:

- NRZ-I;
- Differential Manchester.

Bandingkan hasil encoding dari data yang sama.

14. Ringkasan

1. Line coding merupakan proses merepresentasikan data digital menjadi sinyal digital sesuai aturan tertentu.
2. NRZ menggunakan level yang tidak kembali ke nol selama interval bit. Pada NRZ-L, informasi direpresentasikan melalui level, sedangkan NRZ-I menggunakan perubahan level sebagai bagian dari aturan encoding.
3. RZ mengembalikan sinyal ke level nol di dalam setiap interval bit sehingga menghasilkan lebih banyak transisi.
4. Manchester selalu mempunyai transisi di tengah setiap interval bit. Hal tersebut membantu sinkronisasi antara sender dan receiver.
5. Differential Manchester juga memiliki transisi di tengah bit, tetapi informasi ditentukan berdasarkan perubahan relatif, bukan hanya arah level absolut.
6. Teknik dengan lebih banyak transisi biasanya memberikan keuntungan sinkronisasi, tetapi dapat membutuhkan bandwidth yang lebih besar.
7. Pada modul ini Anda memperkuat kemampuan Python menggunakan:

   `for`, `if`, `else`, `.append()`, `.extend()`, `list`, `def`, `return`, serta visualisasi menggunakan `plt.step()`.
8. Pada semester project, Anda menambahkan **Line Coding Visualizer** yang menjadi dasar untuk mensimulasikan proses pengiriman bit pada modul selanjutnya.

15. Tes Pemahaman

A. Konseptual

1. Apa yang dimaksud line coding?
2. Mengapa data binary perlu diubah menjadi sinyal digital?
3. Apa perbedaan prinsip NRZ-L dan NRZ-I?
4. Mengapa rangkaian bit yang sama dapat menjadi masalah pada NRZ?
5. Apa karakteristik utama RZ?
6. Mengapa Manchester disebut mempunyai kemampuan sinkronisasi yang baik?
7. Apa perbedaan Manchester dan Differential Manchester?
8. Mengapa banyaknya transisi dapat memengaruhi kebutuhan bandwidth?

B. Analisis

Diberikan data:

`00000000`

Bandingkan secara konseptual bagaimana NRZ-L dan Manchester merepresentasikan pola tersebut.

Manakah yang menghasilkan lebih banyak transisi? Jelaskan dampaknya terhadap sinkronisasi.

C. Python

Perhatikan:

```
bits = [1, 0, 1]

levels = []

for bit in bits:
    if bit == 1:
        levels.append(1)
    else:
        levels.append(-1)
```

```
print(levels)
```

Jawab sebelum menjalankan program:

1. apa isi levels?
2. apa fungsi for?
3. apa fungsi if?
4. apa fungsi .append()?

16. Tugas Mandiri

Kasus: Digital Sensor Encoder

Sebuah sensor menghasilkan data:

`10100110`

Buat program Python yang dapat:

1. menerima pola bit dari pengguna;
2. mengubah input menjadi list integer;
3. menghasilkan NRZ-L;
4. menghasilkan Manchester;
5. menampilkan kedua hasil dalam bentuk grafik;
6. menjelaskan perbedaan jumlah transisi yang terlihat.

Gunakan minimal:

```
input()
for
if
.append()
.extend()
def
return
plt.step()
```

Lakukan pengujian menggunakan minimal tiga pola:

- pola bergantian;
- rangkaian `1` panjang;
- rangkaian `0` panjang.

Berikan analisis singkat mengenai teknik mana yang memberikan transisi lebih teratur.

17. Menuju Modul 6

Line coding memungkinkan data digital direpresentasikan menjadi bentuk sinyal.

Namun meskipun sender telah menggunakan encoding yang benar dan media transmisi telah tersedia, sinyal yang tiba di receiver belum tentu identik dengan sinyal yang dikirim.

Selama transmisi dapat terjadi:

- attenuation;
- distortion;
- noise;
- perubahan bit.

Bagaimana kita mengetahui bahwa bit telah berubah?

Bagaimana tingkat kesalahan transmisi dapat diukur?

18. Referensi

Forouzan, B. A. (2013). *Data Communications and Networking* (5th ed.). McGraw-Hill Education.

Stallings, W. (2022). *Data and Computer Communications* (10th ed.). Pearson.

Python Software Foundation. *Python Documentation*.

Matplotlib Development Team. *Matplotlib Documentation*.